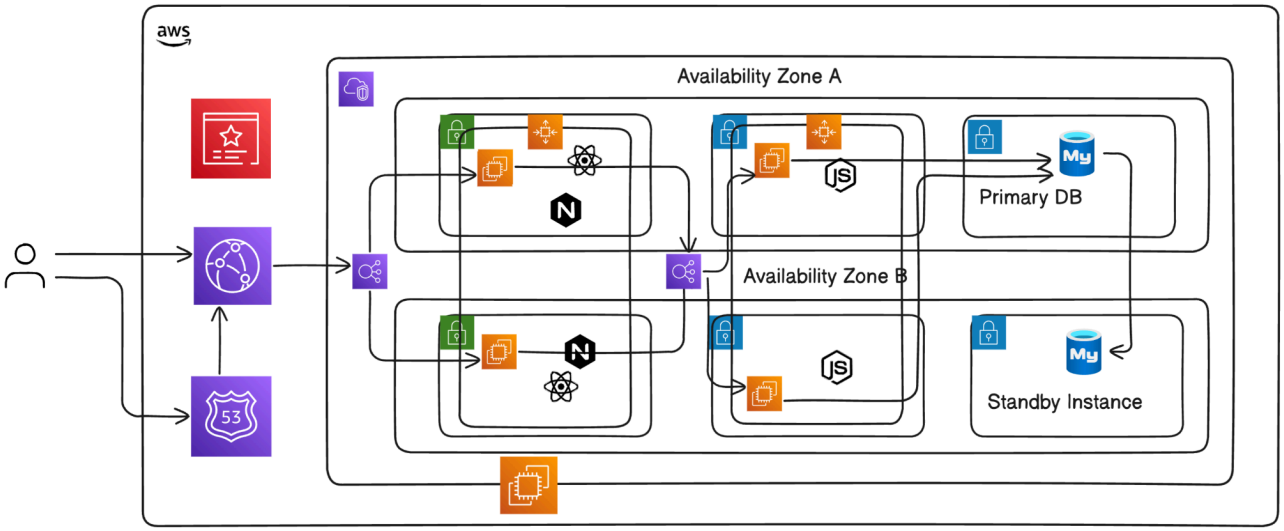
Deploying a Production Grade Highly Available & Scalable 3 Tier Architecture in AWS



Step 1: Configuring Route53 (Optional)

Go to Route53 in AWS

- Click on “Create Hosted Zone”
- Give “Domain name” and Select Public Hosted Zone
- Once Created, You will get NS records
- Add those NS records in DNS of your Domain Provider (Our Case, GoDaddy)
- It will take more than an hour to propagate

Step 2: Requesting a Public SSL Certificate using Amazon ACM

Go to Certificate Manger in AWS

- Click on “Request a Certificate”
- Give your Domain Name, (Ex: devopsdost.xyz www.devopsdost.xyz)
- Once Created, Click on “Create Records in Route53”
- Certificate will be issues

Step 3: Creating VPC & Subnets

Go to VPC Dashboard in AWS

- Click on “Create VPC”
- Click on “VPC and More”
- Name: 3-Tier-Architecture
- Availability Zones: 2
- Public Subnets: 2
- Private Subnets: 4
- Nat Gateway: In 1 AZ
- VPC EndPoints: None
- Click on Create

Will take 2-3 mins to Create

- Go to Subnets, Select Public subnets
- Click on Actions
- Click on Edit Subnet Settings
- Enable Public IP Automatically

Step 4: Creating Security Groups

4.1: Creating security group for Bastion Host

Go to Security groups in EC2 Dashboard

- Click on Create Security Group
- Name: Bastion-Host
- VPC: Select 3-Tier-Architecture
- Inbound Rule: SSH from Anywhere from Ipv4

4.2: Create a Security Group for Presentation Tier ALB

- Click on Create Security Group
- Name: Presentation-Tier-ALB
- VPC: Select 3-Tier Architecture
- Inbound Rule: HTTP from Anywhere from Ipv4

4.3: Creating security Group for Presentation Tier EC2

- Click on Create Security Group
- Name: Presentation-Tier-EC2
- VPC: Select 3-Tier-Architecture
- Inbound Rule:
    - SSH from Bastion Host
    - HTTP from Presentation-Tier-ALB

4.4: Creating Security Group for Application Tier ALB

- Click on Create Security Group
- Name: Application-Tier-ALB
- VPC: Select 3-Tier-Architecture
- Inbound: HTTP from Presentation-Tier-EC2

4.5: Create Security Group for Application Tier EC2

- Click on Create Security Group
- Name: Application-Tier-EC2
- VPC: Select 3-Tier-Architecture
- Inbound Rule:
    - SSH from Bastion Host
    - Custom TCP → 3200 → App-Tier-ALB

4.6: Create Security Group for Data Tier

- Click on Create Security Group
- Name: Data-Tier
- VPC: Select 3-Tier-Architecture
- Inbound Rule:
    - MYSQL/Aurora from Bastion Host
    - MySQL/Aurora from Bastion Host

Step 5: Launching Bastion Host

Go to EC2 Dashboard and click on Launch Instance

- Name: Bastion-Host
- AMI: Select Amazon Linux 2023 AMI
- Instance Type: t2.micro
- KeyPair: Select or Create a New one
- VPC: Select 3-Tier-Architecture
- Subnet: Select Public Subnet
- Security Group: Bastion-Host
- Click on Launch Instance

Step 6: Setup Data Tier with RDS

Go to Amazon RDS in AWS

- Click on Create DB Subnet Group
    - Name: dev-db-subnet-group
    - VPC: 3-Tier-Architecture
    - AZS: Select both
    - Subnets: Select 2 Subnets, Make sure they are private
    - Create Subnet Group
- Click on “Create Database”
    - Standard
    - Select SQL

- Template: dev/test
- Select: Multi AZ DB Instance
- Identifier: dev-db-instance
- Password: Self managed and enter password as “admin123”
- VPC: Select 3-Tier-Architecture
- Select the subnet group which you have created
- Security Group: Data-Tier
- Click on Create Database

It will take 15-20 mins to Complete

Once your Database is Up and running

- Open Command Prompt for Windows/Terminal for MacOS
- Locate the path of your Keypair

To Change SSH key permission

```
chmod 400 your_key.pem
```

To Start SSH Agent

```
eval "$(ssh-agent -s)"
```

To add key to SSH Agent

```
ssh-add your_key.pem
```

To SSH into Bastion Host with Agent forwarding (Optional)

```
ssh -A ec2-user@bastion_host_public_ip
```

Connect to DataBase

```
ssh -N -L 3307:<Database-endpoint>:3306 ec2-user@bastion_host_public_ip
```

Let’s Create Database and Restore the Data

Open MySQL Workbench, You can Install from Internet

- Click on “+”
- Name: rds-dev-db-admin
- Port: 3307
- Username: admin
- Store in Key: admin123
- Test Connection
- If Success, Click on Ok
- Click on that, It will open SQL Editor

Copy and paste and run the below queries

```
-- Create Database and User
CREATE DATABASE react_node_app;
CREATE USER 'appuser'@'%' IDENTIFIED BY 'admin123';
GRANT ALL PRIVILEGES ON react_node_app.* TO ' appuser'@'%';
```

```
FLUSH PRIVILEGES;
```

Go to MySQL Bench Again and click on “+”

- Name: rds-dev-db-appuser
- port: 3307
- username: appuser
- Store in key: admin123
- Test Connection
- If Success, Click on Ok

Click on that to open SQL Editor and run the following Queries:

```
USE react_node_app;

-- Create Tables
CREATE TABLE `author` (
  `id` int NOT NULL AUTO_INCREMENT,
  `name` varchar(255) NOT NULL,
  `birthday` date NOT NULL,
  `bio` text NOT NULL,
  `createdAt` date NOT NULL,
  `updatedAt` date NOT NULL,
  PRIMARY KEY (`id`)
) ENGINE=InnoDB AUTO_INCREMENT=8 DEFAULT CHARSET=utf8mb4
COLLATE=utf8mb4_0900_ai_ci;

CREATE TABLE `book` (
  `id` int NOT NULL AUTO_INCREMENT,
  `title` varchar(255) NOT NULL,
  `releaseDate` date NOT NULL,
  `description` text NOT NULL,
  `pages` int NOT NULL,
  `createdAt` date NOT NULL,
  `updatedAt` date NOT NULL,
  `authorId` int DEFAULT NULL,
  PRIMARY KEY (`id`),
```

```
  KEY `FK_66a4f0f47943a0d99c16ecf90b2` (`authorId`),
  CONSTRAINT `FK_66a4f0f47943a0d99c16ecf90b2` FOREIGN KEY (`authorId`) REFERENCES
`author` (`id`)
) ENGINE=InnoDB AUTO_INCREMENT=10 DEFAULT CHARSET=utf8mb4
COLLATE=utf8mb4_0900_ai_ci;


-- Restore Data
INSERT INTO `author` VALUES (1,'J.K. Rowling (Joanne Kathleen
Rowling)','1965-07-31','J.K. Rowling is a British author best known for writing the
Harry Potter fantasy series. The series has won multiple awards and sold over 500
million copies, becoming the best-selling book series in history. Rowling has also
written other novels, including The Casual Vacancy and the Cormoran Strike crime
series under the pen name Robert Galbraith.','2024-05-29','2024-05-29'),(3,'Jane
Austen','1775-12-16','Jane Austen was an English novelist known for her wit, social
commentary, and romantic stories. Her six major novels, which explore themes of
love, marriage, and money, have earned her a place as one of the greatest writers
in the English language.','2024-05-29','2024-05-29'),(4,'Harper
Lee','1960-07-11','Harper Lee was an American novelist best known for her Pulitzer
Prize-winning novel To Kill a Mockingbird. The novel explores themes of racial
injustice and the importance of compassion. Lee published a sequel, Go Set a
Watchman, in 2015.','2024-05-29','2024-05-29'),(5,'J.R.R.
Tolkien','1954-07-29','J.R.R. Tolkien was a British philologist and writer best
known for his fantasy novels The Hobbit and The Lord of the Rings. Tolkien\'s works
have had a profound influence on the fantasy genre and popular
culture.','2024-05-29','2024-05-29'),(6,'Mary Shelley','1818-03-03','Mary Shelley
was a British novelist, playwright, and short story writer, the daughter of Mary
Wollstonecraft Godwin and the wife of poet Percy Bysshe Shelley. Frankenstein, or,
The Modern Prometheus (1818) is her most famous
work.','2024-05-29','2024-05-29'),(7,'Douglas Adams','1979-10-12','Douglas Adams
was an English science fiction writer, satirist, humorist, dramatist, screenwriter,
and occasional actor. He is best known for the Hitchhiker\'s Guide to the Galaxy
comedy series, which inspired a radio comedy, several books, stage shows, comic
books, a 1981 TV series, a 1984 video game, a 2005 feature film, and a 2008 sequel
film.','2024-05-29','2024-05-29');

INSERT INTO `book` VALUES (1,'Harry Potter and the Sorcerer\'s
Stone','1997-07-26','On his birthday, Harry Potter discovers that he is the son of
two well-known wizards, from whom he has inherited magical powers. He must attend a
famous school of magic and sorcery, where he establishes a friendship with two
young men who will become his companions on his adventure. During his first year at
Hogwarts, he discovers that a malevolent and powerful wizard named Voldemort is in
search of a philosopher\'s stone that prolongs the life of its
owner.',223,'2024-05-29','2024-05-29',1),(3,'Harry Potter and the chamber of
secrets','1998-07-02','Harry Potter and the sophomores investigate a malevolent
threat to their Hogwarts classmates, a menacing beast that hides within the
castle.',251,'2024-05-29','2024-05-29',1),(4,'Pride and Prejudice','1813-01-28','An
```

```
English novel of manners by Jane Austen, first published in 1813. The story centres on the relationships among the Bennet sisters, in particular Elizabeth Bennet the middle daughter, and the wealthy Mr. Darcy. Austen satirizes the social classes of the English gentry through a witty and ironic narrative voice.',224,'2024-05-29','2024-05-29',3),(5,'Harry Potter and the Prisoner of Azkaban','1999-07-08','Harry\'s third year of studies at Hogwarts is threatened by Sirius Black\'s escape from Azkaban prison. Apparently, it is a dangerous wizard who was an accomplice of Lord Voldemort and who will try to take revenge on Harry Potter.',317,'2024-05-29','2024-05-29',1),(6,'Harry Potter and the Goblet of Fire','2000-07-08','Hogwarts prepares for the Triwizard Tournament, in which three schools of wizardry will compete. To everyone\'s surprise, Harry Potter is chosen to participate in the competition, in which he must fight dragons, enter the water and face his greatest fears.',636,'2024-05-29','2024-05-29',1),(7,'The Hitchhiker\'s Guide to the Galaxy','1979-10-12','A comic science fiction comedy series created by Douglas Adams. The story follows the comedic misadventures of Arthur Dent, a hapless Englishman, following the destruction of the Earth by the Vogons, a race of unpleasant bureaucratic aliens. Arthur escapes with his friend Ford Prefect, who reveals himself to be an undercover researcher for the titular Hitchhiker\'s Guide to the Galaxy, a galactic encyclopedia containing information about anything and everything.',184,'2024-05-29','2024-05-29',7),(8,'Frankenstein; or, The Modern Prometheus','1818-03-03','A Gothic novel by Mary Shelley that tells the story of Victor Frankenstein, a young scientist who creates a grotesque creature in an unorthodox scientific experiment. Frankenstein is horrified by his creation and abandons it, but the creature seeks revenge. The novel explores themes of scientific responsibility, creation versus destruction, and the nature of good and evil.',211,'2024-05-29','2024-05-29',6),(9,'The Lord of the Rings: The Fellowship of the Ring','1954-07-29','The first book in J.R.R. Tolkien\'s epic fantasy trilogy, The Lord of the Rings. The Fellowship of the Ring follows hobbit Frodo Baggins as he inherits the One Ring, an evil artifact of power created by the Dark Lord Sauron. Frodo embarks on a quest to destroy the Ring in the fires of Mount Doom, accompanied by a fellowship of eight companions.',482,'2024-05-29','2024-05-29',5);
```

Note: You'll find this code in repository backend folder/db.sql

Step 7: Setup Presentation Tier

7.1 Create Launch Template

Go to Launch Template in EC2 dashboard in AWS

- Name: presentation-tier-lt
- Version: 01
- Enable Auto Scaling Guidance
- Select Amazon Linux in Quick Start
- Instance Type: t2.micro
- Keypair: Select the one you selected for bastion host
- Security Group: Presentation-Tier-EC2
- Go to Advanced → Use Data and paste the below script

```
#!/bin/bash
# Update the package list and install NGINX
sudo yum update -y
sudo yum install nginx -y

# Start and enable NGINX
sudo systemctl start nginx
sudo systemctl enable nginx

# Fetch metadata token
TOKEN=$(curl -X PUT "http://169.254.169.254/latest/api/token" -H
"X-aws-ec2-metadata-token-ttl-seconds: 21600")

# Fetch instance details using IMDSv2
INSTANCE_ID=$(curl -H "X-aws-ec2-metadata-token: $TOKEN"
"http://169.254.169.254/latest/meta-data/instance-id")
AVAILABILITY_ZONE=$(curl -H "X-aws-ec2-metadata-token: $TOKEN"
"http://169.254.169.254/latest/meta-data/placement/availability-zone")
PUBLIC_IP=$(curl -H "X-aws-ec2-metadata-token: $TOKEN"
"http://169.254.169.254/latest/meta-data/public-ipv4")

# Create a simple HTML page displaying instance details
sudo bash -c "cat > /usr/share/nginx/html/index.html <<EOF
<h1>Instance Details</h1>
<p><b>Instance ID:</b> $INSTANCE_ID</p>
<p><b>Availability Zone:</b> $AVAILABILITY_ZONE</p>
```

```
<p><b>Public IP:</b> $PUBLIC_IP</p>
EOF"

# Restart NGINX to ensure changes are applied
sudo systemctl restart nginx
```

7.2 Creating Presentation Tier Target Group

- Click on Target Groups in EC2 Dashboard
- Select Instances
- Name: Presentation-Tier-TG
- VPC: Select 3-Tier-Architecture
- Health Check Path: /health
- Next → Create Target Group

7.3 Creating Presentation Tier Load Balancer

- Click on Load Balancer in EC2 Dashboard
- Click on Create Load Balancer
- Name: Presentation-Tier-ALB
- Type: Internet Facing
- VPC: Select 3-Tier-Architecture
- Subnets: Select Public Subnets
- Security Group: Presentation-Tier-ALB
- Select the Target Group which we created in previous step
- Create Load Balancer

7.4 Creating Presentation Tier Auto Scaling Group

Click on Auto Scaling Group in EC2 Dashboard

- Click on Create ASG
- Name: Presentation-Tier-ASG
- Select Launch Template which we created in the step 7.1
- VPC: 3-Tier-Architecture
- Subnets: Make sure you select public subnets
- Next
- Attach to an Existing Load Balancer → Select Target group “Presentation-Tier-TG”
- Enable Health Checks of Load Balancer
- Enable Cloud Watch Monitoring
- Next
- Desired Capacity: 3
- Minimum: 2
- Maximum: 4
- Select Target Tracking Scaling Policy
- Average CPU Utilization: 90
- Create ASG

If the Configuration is Perfect, It will spin 3 instances in

EC2 Dashboard, and Access the Load Balancer DNS Name.. If you see Instance Metadata Details
Perfect!!!!

Step 8: Setup Application Tier
Similar process like Step 7:

8.1 Create Launch Template

- Name: Application-Tier-lt
- Version: 1
- Enable Auto Scaling Guidance
- Amazon Linux
- T2.micro
- Select Keypair
- Security Group: Application-Tier-EC2
- Go to Advanced → User Data → Copy & paste the below script after making necessary changes!!

Changes:

- Add RDS End Point
- Add DB User as “appuser”
- Add DB Password as “admin123
- Add DB as “react_node_app”

```
#!/bin/bash
# Update package list and install required packages
sudo yum update -y
sudo yum install -y git

# Install Node.js (use NodeSource for the latest version)
curl -fsSL https://rpm.nodesource.com/setup_18.x | sudo bash -
sudo yum install -y nodejs

# Install PM2 globally
sudo npm install -g pm2

# Define variables
REPO_URL="https://github.com/suneelprojects/react-node-mysql-app.git"
BRANCH_NAME="feature/add-logging"
REPO_DIR="/home/ec2-user/react-node-mysql-app/backend"
ENV_FILE="$REPO_DIR/.env"

# Clone the repository
cd /home/ec2-user
sudo -u ec2-user git clone $REPO_URL
cd react-node-mysql-app

# Checkout to the specific branch
sudo -u ec2-user git checkout $BRANCH_NAME
cd backend

# Define the log directory and ensure it exists
LOG_DIR="/home/ec2-user/react-node-mysql-app/backend/logs"
mkdir -p $LOG_DIR
sudo chown -R ec2-user:ec2-user $LOG_DIR

# Append environment variables to the .env file
echo "LOG_DIR=$LOG_DIR" >> "$ENV_FILE"
echo "DB_HOST=\"<rds-instance.end.point.region.rds.amazonaws.com>\"" >> "$ENV_FILE"
echo "DB_PORT=\"3306\"" >> "$ENV_FILE"
echo "DB_USER=\"<db-user>\"" >> "$ENV_FILE"
echo "DB_PASSWORD=\"<db-user-password>\"" >> "$ENV_FILE"  # Replace with actual
password
echo "DB_NAME=\"<db-name>\"" >> "$ENV_FILE"

# Install Node.js dependencies as ec2-user
sudo -u ec2-user npm install

# Start the application using PM2 as ec2-user
sudo -u ec2-user npm run serve
```

```
# Ensure PM2 restarts on reboot as ec2-user
sudo -u ec2-user pm2 startup systemd
sudo -u ec2-user pm2 save
```

8.2 Create Target Group

- Click on Create Target Group → Instances
- Name: Application-Tier-TG
- port: 3200
- VPC: 3-Tier
- Health Check Path: /health
- Create Target Group

8.3 Creating Application Tier Load Balancer

- Click on Create Load Balancer
- Name: Application-Tier-ALB
- Scheme/Type: Internal
- VPC: 3-Tier-Architecture
- Subnets: Select private
- Security Group: Application-Tier-ALB
- Target Group: Select Application-Tier-TG
- Create Load Balancer

8.4 Creating Auto Scaling Group for Application Tier

- Click on Create Auto Scaling Group
- Select Launch Template: Application-Tier-lt
- VPC: 3-Tier-Architecture
- Subnets: Private
- Load Balancer: Existing → Select "Application-Tier-TG"
- Enable Health Checks for Load Balancer
- Enable CloudWatch Monitoring
- Next
- Desired: 3
- Minimum: 4
- Maximum: 4
- Target Tracking Scaling Policy
- Average Tracking Scaling Policy → 90
- Create ASG

If Configured Correctly, Another 3 instances will be created in EC2

To Check whether Backend is working or not,

Connect to Bastion Host from Terminal:

```
ssh -A ec2-user@bastion_host_public_ip
```

Login to Private Instance of Application Tier

```
ssh ec2-user@private-ip
```

Once Connected, Run the following command

```
pm2 logs
```

If t shows logs, Your Backend is setup perfectly!!!

Step 9: Modifying Presentation Tier Launch Template

Now, we will deploy our ReactJS Frontend!!

9.1 Modifying Launch Template

- Go to Launch Templates
- Select Presentation-Tier-lt
- Actions → Modify

- Version: 2
- Go to Advanced → User Data → Copy and paste the below script with necessary changes

Changes to do:

- Add Application Tier DNS Name
- Add Domain names

```
#!/bin/bash
# Update package list and install required packages
sudo yum update -y
sudo yum install -y git

# Install Node.js (use NodeSource for the latest version)
curl -fsSL https://rpm.nodesource.com/setup_18.x | sudo bash -
sudo yum install -y nodejs

# Install NGINX
sudo yum install -y nginx

# Start and enable NGINX
sudo systemctl start nginx
sudo systemctl enable nginx

# Define variables
REPO_URL="https://github.com/suneelprojects/react-node-mysql-app.git"
BRANCH_NAME="feature/add-logging"
REPO_DIR="/home/ec2-user/react-node-mysql-app/frontend"
ENV_FILE="$REPO_DIR/.env"
APP_TIER_ALB_URL="http://<internal-application-tier-alb-end-point.region.elb.amazonaws.com>"  # Replace with your actual alb endpoint
API_URL="/api"

# Clone the repository as ec2-user
cd /home/ec2-user
sudo -u ec2-user git clone $REPO_URL
cd react-node-mysql-app
```

```
# Checkout to the specific branch
sudo -u ec2-user git checkout $BRANCH_NAME
cd frontend

# Ensure ec2-user owns the directory
sudo chown -R ec2-user:ec2-user /home/ec2-user/react-node-mysql-app

# Create .env file with the API_URL
echo "VITE_API_URL=\"$API_URL\"" >> "$ENV_FILE"

# Install Node.js dependencies as ec2-user
sudo -u ec2-user npm install

# Build the frontend application as ec2-user
sudo -u ec2-user npm run build

# Copy the build files to the NGINX directory
sudo cp -r dist /usr/share/nginx/html/

# Update NGINX configuration
NGINX_CONF="/etc/nginx/nginx.conf"
SERVER_NAME="<domain subdomain>"  # Replace with your actual domain name

# Backup existing NGINX configuration
sudo cp $NGINX_CONF ${NGINX_CONF}.bak

# Write new NGINX configuration
sudo tee $NGINX_CONF > /dev/null <<EOL
user nginx;
worker_processes auto;

error_log /var/log/nginx/error.log warn;
pid /run/nginx.pid;

events {
    worker_connections 1024;
}

http {
    include /etc/nginx/mime.types;
    default_type application/octet-stream;

    log_format main '\$remote_addr - \$remote_user [\$time_local] "\$request" '
                    '\$status \$body_bytes_sent "\$http_referer" '
                    '"\$http_user_agent" "\$http_x_forwarded_for"';

    access_log /var/log/nginx/access.log main;
```

```
    sendfile on;
    tcp_nopush on;
    tcp_nodelay on;
    keepalive_timeout 65;
    types_hash_max_size 2048;

    include /etc/nginx/conf.d/*.conf;
}
EOL

# Create a separate NGINX configuration file
sudo tee /etc/nginx/conf.d/presentation-tier.conf > /dev/null <<EOL
server {
    listen 80;
    server_name $SERVER_NAME;
    root /usr/share/nginx/html/dist;
    index index.html index.htm;

    #health check
    location /health {
        default_type text/html;
        return 200 "<!DOCTYPE html><p>Health check endpoint</p>\n";
    }

    location / {
        try_files \$uri /index.html;
    }

    location /api/ {
        proxy_pass $APP_TIER_ALB_URL;
        proxy_set_header Host \$host;
        proxy_set_header X-Real-IP \$remote_addr;
        proxy_set_header X-Forwarded-For \$proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto \$scheme;
    }
}
EOL


# Restart NGINX to apply the new configuration
sudo systemctl restart nginx
```

- Click on Update

9.2 Modifying Auto Scaling Group of Presentation Tier

- Click on Edit
- Desired: 2
- Min: 2
- Max: 2
- Select Launch Template Version: 2
- Done

Terminate Old Instances to get new instances with new configuration

Now You can Access the Application using DNS Name of Presentation Tier Load Balancer

Step 10: Integrating Application Logs with CloudWatch

- Connect one of our App Tier Ec2 instances
- ls
- Go to Repo Directory/bakcend/logs
- vim combined.log
- You can See logs

Follow this Process

10.1 Creating IAM Role

Go to IAM Dashboard → Roles → Create → EC2 → “CloudwatchLogsFullAccess”
“CloudWatchAgentServerPolicy” → Next → Give name as “EC2InstanceRoleForCloudWatchLogs” → Create

10.2 Creating Cloud Watch Log Group

Go to CloudWatch → Log Groups → Create Log Group → Name: backend-node-app-logs → Create

Step 11: Modifying Application Tier Launch Template

- Go to Launch Templates in EC2
- Select Application-Tier-lt
- Select Version 2
- Go to Advanced Details
- Select IAM Instance Profile → Attach Role We created
- Enable Detailed CloudWatch Monitoring
- User Data → Add the Below Script

```
#!/bin/bash
# Update package list and install required packages
sudo yum update -y
```

```
sudo yum install -y git

# Install Node.js (use NodeSource for the latest version)
curl -fsSL https://rpm.nodesource.com/setup_18.x | sudo bash -
sudo yum install -y nodejs

# Install NGINX
sudo yum install -y nginx

# Start and enable NGINX
sudo systemctl start nginx
sudo systemctl enable nginx

# Define variables
REPO_URL="https://github.com/suneelprojects/react-node-mysql-app.git"
BRANCH_NAME="feature/add-logging"
REPO_DIR="/home/ec2-user/react-node-mysql-app/frontend"
ENV_FILE="$REPO_DIR/.env"
APP_TIER_ALB_URL="http://<internal-application-tier-alb-end-point.region.elb.amazon
aws.com>"  # Replace with your actual alb endpoint
API_URL="/api"

# Clone the repository as ec2-user
cd /home/ec2-user
sudo -u ec2-user git clone $REPO_URL
cd react-node-mysql-app

# Checkout to the specific branch
sudo -u ec2-user git checkout $BRANCH_NAME
cd frontend

# Ensure ec2-user owns the directory
sudo chown -R ec2-user:ec2-user /home/ec2-user/react-node-mysql-app

# Create .env file with the API_URL
echo "VITE_API_URL=\"$API_URL\"" >> "$ENV_FILE"

# Install Node.js dependencies as ec2-user
sudo -u ec2-user npm install

# Build the frontend application as ec2-user
sudo -u ec2-user npm run build

# Copy the build files to the NGINX directory
sudo cp -r dist /usr/share/nginx/html/

# Update NGINX configuration
```

```
NGINX_CONF="/etc/nginx/nginx.conf"
SERVER_NAME="<domain subdomain>"  # Replace with your actual domain name

# Backup existing NGINX configuration
sudo cp $NGINX_CONF ${NGINX_CONF}.bak

# Write new NGINX configuration
sudo tee $NGINX_CONF > /dev/null <<EOL
user nginx;
worker_processes auto;

error_log /var/log/nginx/error.log warn;
pid /run/nginx.pid;

events {
    worker_connections 1024;
}

http {
    include /etc/nginx/mime.types;
    default_type application/octet-stream;

    log_format main '\$remote_addr - \$remote_user [\$time_local] "\$request" '
                    '\$status \$body_bytes_sent "\$http_referer" '
                    '"\$http_user_agent" "\$http_x_forwarded_for"';

    access_log /var/log/nginx/access.log main;

    sendfile on;
    tcp_nopush on;
    tcp_nodelay on;
    keepalive_timeout 65;
    types_hash_max_size 2048;

    include /etc/nginx/conf.d/*.conf;
}
EOL

# Create a separate NGINX configuration file
sudo tee /etc/nginx/conf.d/presentation-tier.conf > /dev/null <<EOL
server {
    listen 80;
    server_name $SERVER_NAME;
    root /usr/share/nginx/html/dist;
    index index.html index.htm;

    #health check
```

```
    location /health {
        default_type text/html;
        return 200 "<!DOCTYPE html><p>Health check endpoint</p>\n";
    }

    location / {
        try_files \$uri /index.html;
    }

    location /api/ {
        proxy_pass $APP_TIER_ALB_URL;
        proxy_set_header Host \$host;
        proxy_set_header X-Real-IP \$remote_addr;
        proxy_set_header X-Forwarded-For \$proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto \$scheme;
    }
}
EOL


# Restart NGINX to apply the new configuration
sudo systemctl restart nginx

# Install CloudWatch agent
sudo yum install -y amazon-cloudwatch-agent

# Create CloudWatch agent configuration
sudo tee /opt/aws/amazon-cloudwatch-agent/etc/amazon-cloudwatch-agent.json >
/dev/null <<EOL
{
  "logs": {
    "logs_collected": {
      "files": {
        "collect_list": [
          {
            "file_path": "/home/ec2-user/react-node-mysql-app/backend/logs/*.log",
            "log_group_name": "backend-node-app-logs",
            "log_stream_name": "{instance_id}",
            "timestamp_format": "%Y-%m-%d %H:%M:%S"
          }
        ]
      }
    }
  }
}
EOL
```

```
# Start CloudWatch agent
sudo /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-ctl -a
fetch-config -m ec2 -c
file:/opt/aws/amazon-cloudwatch-agent/etc/amazon-cloudwatch-agent.json -s
```

- Update

11.1 Modifying ASG

- Click on Application-Tier-ASG → Edit
- Version 2
- Update

To see changes, Terminate the Old Instances, New instances will get created

Access the Application → Add Book → Go to Cloud Watch Log Group → Check Logs

Step 12: Creating CloudFront Distribution

Go to CloudFront → Create → Origin Domain → Presentation-Tier-ALB → HTTP → Redirect HTTP to HTTPS → Do not Enable WAF → Select needed regions → Add Alternate Domain name → “devopsdost.xyz” “www.devopsdost.xyz” → Attach SSL Certificate →

Create Distribution

Step 13: Create DNS Records fro CloudFront in Route53

Go to Route53 → Open Our Hosted Zone → Click on Add Record → Alias → To Cloud Front Distribution → Select your distribution

Do the Same for www.devopsdost.com also

Step 14: Test the Application

Access the Domain, You can see the Application

FULLY FUNCTIONAL, HIGHLY AVAILABLE, SCALABLE THREE TIER ARCHITECTURE ON AWS